

American University of Iraq - Sulaimani

Hospital Emergency Department System

Total marks: 10 points

Language: Java

You are building a patient management system for a hospital emergency department (ED). Patients arrive at the ED at different times, are assigned a **triage severity level** (1 = critical, 2 = urgent, 3 = less urgent, 4 = non-urgent), and wait to be seen by a doctor. The system must track the waiting list, sort patients by severity or wait time so doctors can make informed decisions quickly.

Each patient in the system is represented as an object with the following fields:

Patient object:

int patientID: unique identifier e.g. 1001 String
name: patient full name
int severity: triage level: 1 (critical) → 4 (non-urgent)
int waitTime: minutes waiting since arrival

The emergency department list is a list of patients ordered by arrival time. Implement the following operations:

1. **admitPatient(id, name, severity, waitTime):** add a new patient to the end of the patient list. (they join the back of the line). **(1 Point)**
2. **dischargeNext():** remove and return the very first patient of patient list (the longest-waiting patient is seen first). Print: **"Doctor is now seeing: [name] (ID: [id])"** **(1 Point)**
3. **upgradeToFront(patientID):** find a patient anywhere in the queue by their ID and move them to the head. This simulates a nurse escalating a patient whose condition has worsened. If the ID is not found, print an appropriate error message. **(2 Points)**
4. **removePatient(patientID):** remove a patient by ID from any position (they left without being seen). Update all pointers correctly. **(1 Point)**
5. **searchPatient(patientID):** find and print a patient's full details by ID, or report not found. **(1 Point)**
6. **printList():** print all recorded patients in the format: **(1 Point)**

[1001: Ali, sev=2, wait=30min] → [1002: Sara, sev=1, wait=15min] → NULL

Test scenario:

admitPatient(1001, "Ali Hassan", 2, 30)
admitPatient(1002, "Sara Ahmed", 1, 15)
admitPatient(1003, "Omar Karimi", 3, 45)
admitPatient(1004, "Lena Noor", 2, 10)
printQueue() → show all 4 patients
upgradeToFront(1003) → Omar moves to head
printQueue() → verify Omar is first
dischargeNext() → Omar is seen
removePatient(1004) → Lena left without being seen
printList() → show remaining patients

7. At the top of every hour, the charge nurse re-sorts the waiting queue so that the most critical patients (severity = 1) are at the head, and the least urgent (severity = 4) are at the tail. Implement **sortBySeverity()** using one of the sorting techniques covered in class. In each pass, find the node with the lowest severity value in the unsorted portion and swap its data with the current node. Do not create new nodes — only swap field values. **(3 Points)**

⇒ If two patients have the same severity level, the one with the higher waitTime should come first (they have waited longer).

Test input:

Patient list before sort:

[1001: Ali, sev=2, wait=30] → [1002: Sara, sev=1, wait=15] → [1005: Yusuf, sev=3, wait=50]
→ [1006: Mia, sev=1, wait=40] → NULL

Expected after sort (sev=1 tie broken by wait time):

[1006: Mia, sev=1, wait=40] → [1002: Sara, sev=1, wait=15] → [1001: Ali, sev=2, wait=30] →
[1005: Yusuf, sev=3, wait=50] → NULL

SUBMISSION CHECKLIST

- Name each source file clearly.
- Include a comment block at the top of every file: name, student ID, and date.
- Code must compile and run without errors. A file that does not compile receives zero.
- Submit the source files (.java files). Do not submit compiled binaries, IDE project folders, or screenshots of output.

NOTE: No built-in data structures or sorting functions are allowed. This includes ArrayList, LinkedList, Collections.sort(), Arrays.sort(), list.sort(), sorted(), std::list, std::sort, or any equivalent. All structures and algorithms must be implemented from scratch using your own Node and list classes.